BootCamp_02 – Double Linked Lists (Optional)

Student Information

Integrity Policy: All university integrity and class syllabus policies have been followed. I have neither given, nor received, nor have I tolerated others' use of unauthorized aid.

I understand and followed these policies: ◯ Yes ◯ No

Name:

Date:

Submission Details

Final ***Changelist*** number:

Verified build: ◯ Yes ◯ No

(75% max) PA to be regraded:

Required Configurations: DEBUG only

Discussion (What did you learn):

Verify Builds

- Follow the Piazza procedure on submission
  - Verify your submission compiles and works at the changelist number.
- Verify that only MINIMUM files are submitted
  - No – Generated files
    - *.pdb, *.suo, *.sdf, *.user, *.obj, *.exe, *.log, *.pdb, *.db, *.user
    - Anything that is generated by the compiler should not be included
  - No – Generated directories
    - /Debug, /Release, /Log, /ipch, /.vs
- Typical files project files that are required
  - *.sln, *.cpp, *.h
  - *.vcxproj, *.vcxproj.filters, CleanMe.bat

Standard Rules

Submit multiple times to Perforce

- Submit your work as you go to perforce several times (at least 5)
  - As soon as you get something working, submit to perforce
  - Have reasonable check-in comments
    - Points will be deducted if minimum is not reached

Write all programs in cross-platform C++

- Optimize for execution speed and robustness
- Working code doesn't mean full credit

Submission Report

- Fill out the submission Report
  - No report, no grade

Code and project needs to compile and run

- Make sure that your program compiles and runs
  - Warning level ALL …
  - NO Warnings or ERRORS
    - Your code should be squeaky clean.
  - Code needs to work "as-is".
    - No modifications to files or deleting files necessary to compile or run.
  - All your code must compile from perforce with no modifications.
    - Otherwise it's a 0, no exceptions

Project needs to run to completion

- If it crashes for any reason…
  - It will not be graded and you get a 0

No Containers

- NO STL allowed {Vector, Lists, Sets, etc...}
  - No automatic containers or arrays
  - You need to do this the old fashion way - ***YOU EARNED IT***

Leave Project Settings

- Do NOT change the project or warning level
  - Any changing of level or suppression of warnings is an integrity issue

Simple C++

- No modern C++
  - No Lambdas, Autos, templates, etc…
  - No Boost
- NO Streams
  - Used fopen, fread, fwrite…
- No code in MACROS
  - Code needs to be in cpp files to see and debug it easy
- ***Exception:***
  - implicit problem needs templates

Leaking Memory

- If the program leaks memory
  - There is a deduction of 20% of grade
- If a class creates an object using new/malloc
  - It is responsible for its deletion
- Any ***MEMORY*** dynamically allocated that isn't freed up is ***LEAKING***
  - Leaking is ***HORRIBLE***, so you lose points

No Debug code or files disabled

- Make sure the program is returned to the original state
  - If you added debug code, please return to original state
- If you disabled file, you need to re-enable the files
  - All files must be active to get credit.
  - Better to lose points for unit tests than to disable and lose all points

No Adding files to this project

- This project will work “as-is” do not add files…
- Grading system will overwrite project settings and will ignore any student’s added files and will returned program to the original state

UnitTestConfiguration file (if provided) needs to be set by user

- Grading will be on the UnitTestConfiguration settings
  - Please explicitly set which tests you want graded… no regrading if set incorrectly

Due Dates

- ~~See Piazza for due date and time~~
- Submit program perforce in your student directory assignment supplied.
- Fill out your this ***Submission Report*** and commit to perforce
    - ***ONLY*** use Adobe Reader to fill out form, all others will be rejected.
    - Fill out the form and discussion for full credit.

Goals

- Single Linked list
    - Fundamentals
- Increasing C++ knowledge and understanding

Assignments

- Write a double linked list
    - Add to front of list
    - Add to end of list
    - Remove any node
    - Print the contents of the list
- Simple double linked list
    - No tail pointers

```
class Car
{
public:

	// do not add data
	Car *pNext;
	Car *pPrev;
	const char *pName;
};

class Garage
{
public:

	// do not add data
	Car *pHead;
};
```

- Have your output MATCH this sample

```
**************************************
**      Framework: 3.82               **
**   C++ Compiler: 193732822          **
**  Tools Version: 14.37.32822        **
**    Windows SDK: 10.0.22621.0       **
**   Mem Tracking: enabled            **
**           Mode: x86 Debug          **
**************************************

--------------------------------
    Memory Tracking: start()
--------------------------------

Garage(Add to Front): head:Escape
   Car(Escape)
    next--->Outback
    prev--->null
   Car(Outback)
    next--->Forte
    prev--->Escape
   Car(Forte)
    next--->Accord
    prev--->Outback
   Car(Accord)
    next--->Jetta
    prev--->Forte
   Car(Jetta)
    next--->null
    prev--->Accord

Garage(Remove E (Escape)): head:Outback
   Car(Outback)
    next--->Forte
    prev--->null
   Car(Forte)
    next--->Accord
    prev--->Outback
   Car(Accord)
    next--->Jetta
    prev--->Forte
   Car(Jetta)
    next--->null
    prev--->Accord

Garage(Remove C (Forte)): head:Outback
   Car(Outback)
    next--->Accord
    prev--->null
   Car(Accord)
    next--->Jetta
    prev--->Outback
   Car(Jetta)
    next--->null
    prev--->Accord

Garage(Remove A (Jetta)): head:Outback
   Car(Outback)
```

```
    next--->Accord
    prev--->null
   Car(Accord)
    next--->null
    prev--->Outback

Garage(Remove B (Accord)): head:Outback
   Car(Outback)
    next--->null
    prev--->null

Garage(Remove D (Outback)): head:null

Garage(Add to End A (Jetta)): head:Jetta
   Car(Jetta)
    next--->null
    prev--->null

Garage(Add to End B (Accord)): head:Jetta
   Car(Jetta)
    next--->Accord
    prev--->null
   Car(Accord)
    next--->null
    prev--->Jetta

Garage(Add to End C (Forte)): head:Jetta
   Car(Jetta)
    next--->Accord
    prev--->null
   Car(Accord)
    next--->Forte
    prev--->Jetta
   Car(Forte)
    next--->null
    prev--->Accord

Garage(Add to End D (output)): head:Jetta
   Car(Jetta)
    next--->Accord
    prev--->null
   Car(Accord)
    next--->Forte
    prev--->Jetta
   Car(Forte)
    next--->Outback
    prev--->Accord
   Car(Outback)
    next--->null
    prev--->Forte

Garage(Remove A,B,C,D): head:null


--------------------------------
    Memory Tracking: passed
--------------------------------
    Memory Tracking: end()
--------------------------------
```

- Use the main() provided
    - If you complete this assignment and your output looks like sample
    - Submit it
        - Fill in the PDF as usual
        - Fill in the box on what PA you want regraded
            - Max 75% PA grade will be given
        - Private piazza me that your BootCamp_01 is done
            - I can added it to the regrade list
        - BootCamp is 100% optional